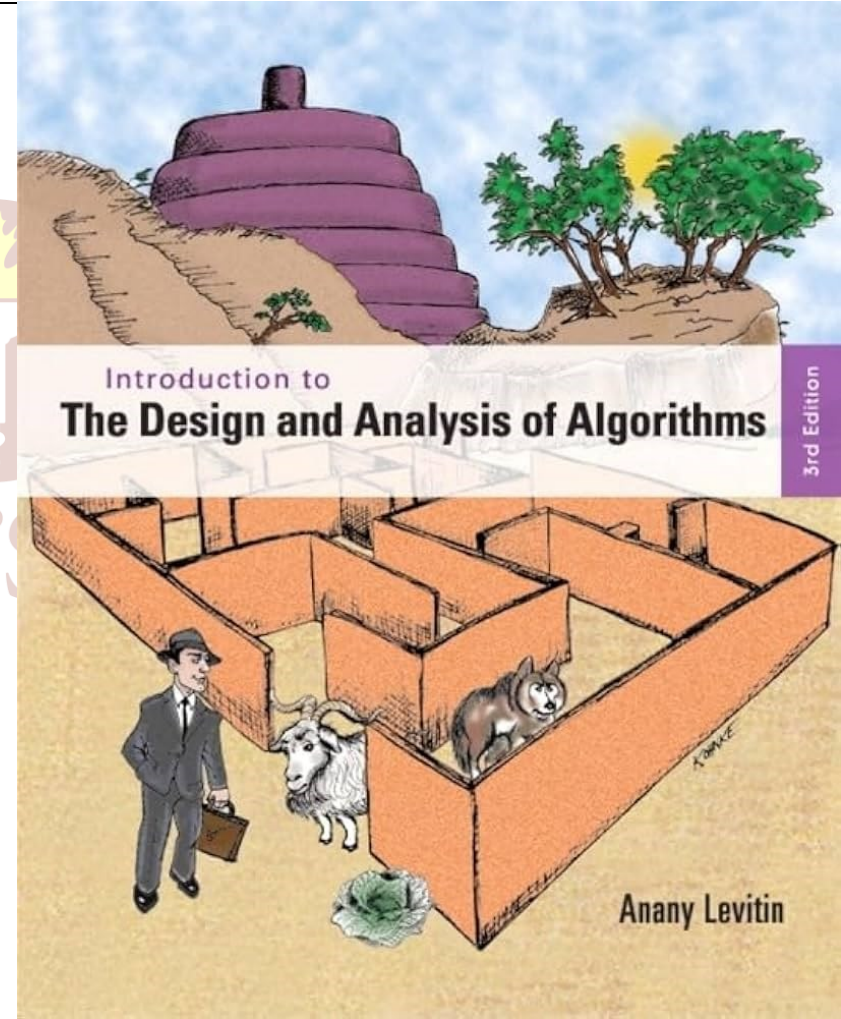


CS 07340 DESIGN AND ANALYSIS OF ALGORITHMS

WEEK 7 OVERVIEW

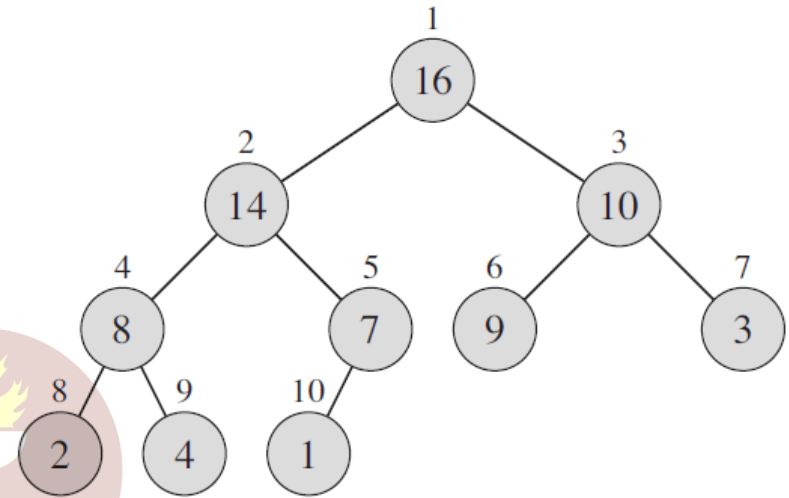
- Transform and Conquer
 - Heaps and Heapsort
 - Binary Exponentiation
- Space and Time Trade-Offs
 - Hashing and Hash Tables



TRANSFORM AND CONQUER

HEAPS

A **binary heap** is a nearly complete binary tree filled on all levels except possibly the lowest level where leaves (key-values) are pushed left-most. Heaps are usually implemented as an array.

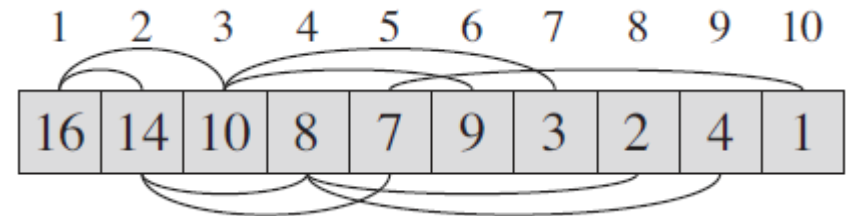


Each node in a heap satisfies the **heap property**:

max-heap: Every node's value is larger than the values of its children.

min-heap: Every node's value is less than the value of its children.

We will focus on creating proper heaps.



RELATIONSHIP BETWEEN ADT PRIORITY QUEUE AND MIN-HEAP

```
class Node {
    int key;
    int value;
    Node leftChild;
    Node rightChild;
}

class Heap {
    private Node heapArray[];

    public void insert(Node node) {...}
    public Node remove() {...}
}

class priorityQueue {
    private Heap qHeap;

    public void insert(Node node) {
        qHeap.insert(node);
    }
    public Node remove() {
        return qHeap.remove();
    }
}
```



A priority queue is an ADT which has methods for inserting an item and removing the smallest (or largest) item. As seen, the methods for class **priorityQueue** are just wrappers for the underlying **Heap** class. A priority queue can be implemented in several ways – a heap being one – but a heap is a more fundamental data structure.



MIN-HEAP METHODS OVERVIEW

Min (H)

- Return the value of the root element
- Runtime $O(1)$

Insert (H, k)

- Place element **k** into the right-most position of the last level
- Move up node until it satisfied the (min) heap property
- Runtime $O(\log(n))$

DeleteMin (H)

- Remove root
- Place the right-most leaf into the root position
- Move down node to satisfy the heap property
- Runtime $O(\log(n))$

MAX-HEAP METHODS OVERVIEW

Max (H)

- Return the value of the root element
- Runtime $O(1)$

Insert (H, k)

- Place element **k** into the right-most position of the last level
- Move up node until it satisfied the (max) heap property
- Runtime $O(\log(n))$

DeleteMax (H)

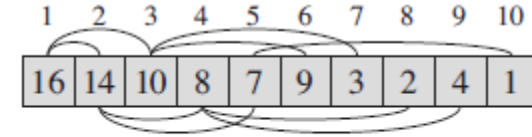
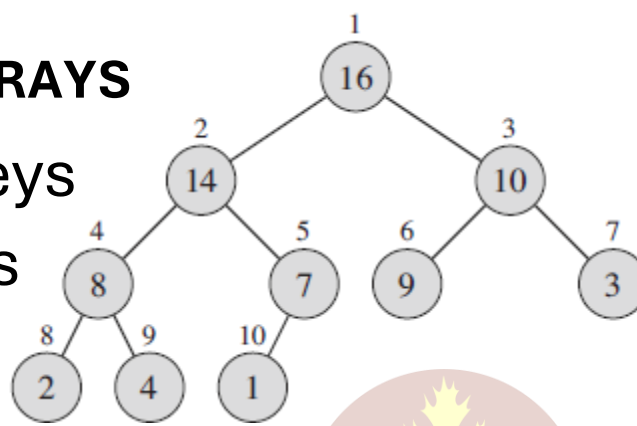
- Remove root
- Place the right-most leaf into the root position
- Move down node to satisfy the heap property
- Runtime $O(\log(n))$

HEAPS IMPLEMENTED AS ARRAYS

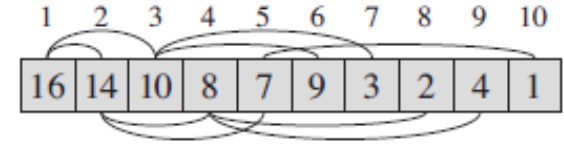
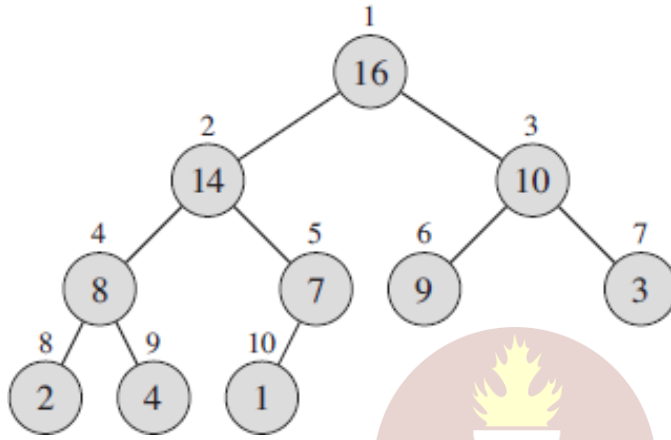
An array A of integer keys that represents a heap has two additional attributes (compared to an array):

- $A.length$
 - Number of elements the array could hold
- $A.heap\text{-}size$
 - Number of elements stored in the heap

Note that indices start from 1 instead of zero, and node entries are integer keys and not values. The root of the tree is $A[1]$. Entries outside $1 \dots A.heap\text{-}size$ are just ignored (no garbage collection).



A.length should be the nearest power of two when rounding up from A.heap-size to accommodate insertion of nodes.



$$A.length = 2^{\lceil \log_2(A.heap-size) \rceil}$$

TREE TO ARRAY

Because binary trees (where each node has at most two children, usually called **left** and **right**) are so regular in their structure, we can represent heaps in an array without links or references.

Parent(*i*)

return $\lfloor i/2 \rfloor$

Left(*i*)

return $2i$

Right(*i*)

return $2i + 1$

Determining position is $O(1)$.



HEAP PROPERTY

The ***heap property*** is what allows operations to be performed quickly. Since we want to be able to find the minimum (or maximum) quickly, we want the minimum (or maximum) to be the root, and any node should be smaller (or larger) than its descendants.

max-heap

- $A[\text{Parent}(i)] > A[i]$

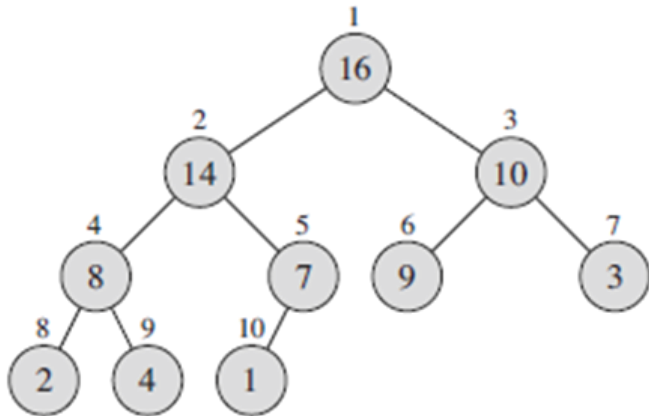
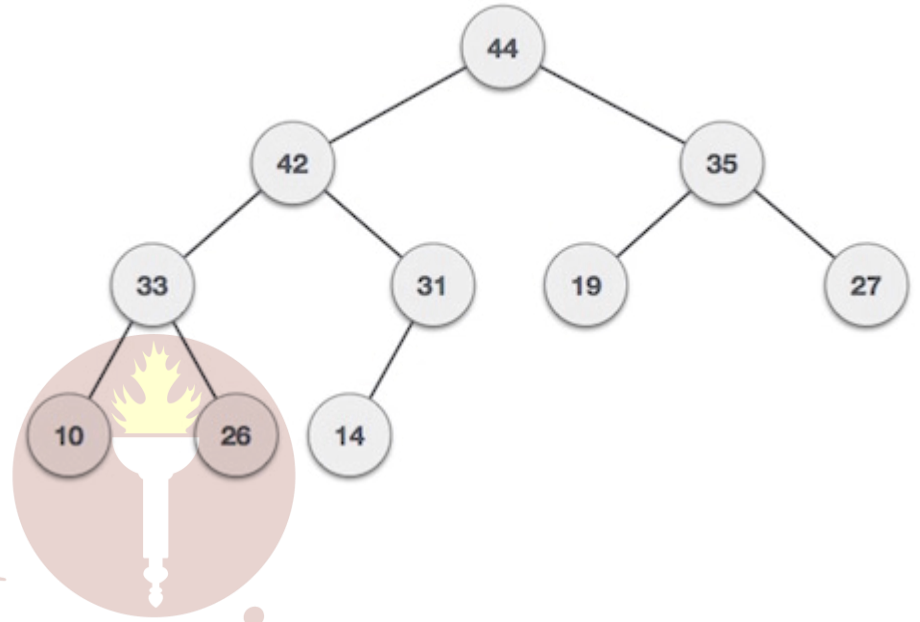
min-heap

- $A[\text{Parent}(i)] < A[i]$

Where is the maximum in a max-heap? Where is the minimum?

The largest element in a max-heap is stored at the root. **The minimum is not necessarily the last element in a max heap!**

The smallest element in a min-heap is at the root.



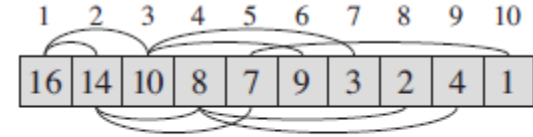
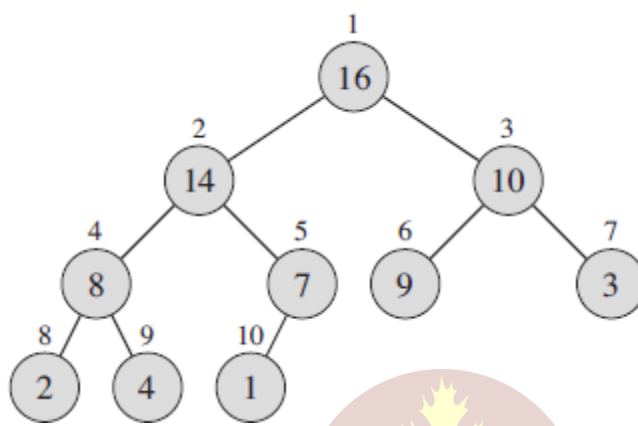
EXAMPLE

Access calculations:

- $\text{Node}(2) = A[2] = 14$
- $\text{Parent}(2) = \lfloor 2/1 \rfloor = 1$
- $\text{Node}(1) = 16$
- $\text{Left}(2) = 2 \cdot 2 = 4$
- $\text{Node}(4) = 8$
- $\text{Right}(2) = 2 \cdot 2 + 1 = 5$
- $\text{Node}(5) = 7$

Heap property:

- $A[\text{Parent}(2)] = A[2] = 14 > A[\text{Left}(2)] = A[4] = 8$
- $A[\text{Parent}(2)] = A[2] = 14 > A[\text{Right}(2)] = A[5] = 7$



Which of the following arrays is ordered as a max heap?

25, 14, 16, 13, 10, 8, 12

25, 12, 16, 13, 10, 8, 14

25, 14, 12, 13, 10, 8, 16

25, 14, 13, 16, 10, 8, 12



ALGORITHM *HeapBottomUp*($H[1..n]$)

HEAP BOTTOM UP

//Constructs a heap from elements of a given array

// by the bottom-up algorithm

//Input: An array $H[1..n]$ of orderable items

//Output: A heap $H[1..n]$

for $i \leftarrow \lfloor n/2 \rfloor$ **downto** 1 **do**

$k \leftarrow i$; $v \leftarrow H[k]$

$heap \leftarrow \mathbf{false}$

while not $heap$ **and** $2 * k \leq n$ **do**

$j \leftarrow 2 * k$

if $j < n$ //there are two children

if $H[j] < H[j + 1]$ $j \leftarrow j + 1$

if $v \geq H[j]$

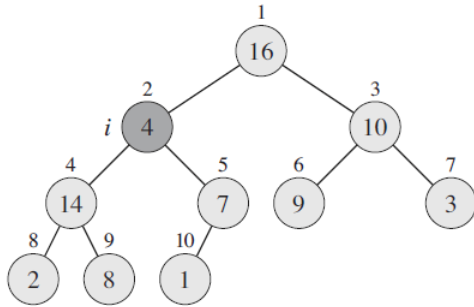
$heap \leftarrow \mathbf{true}$

else $H[k] \leftarrow H[j]$; $k \leftarrow j$

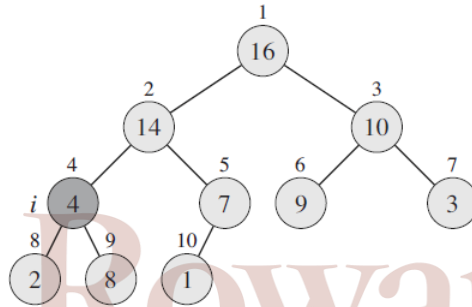
$H[k] \leftarrow v$



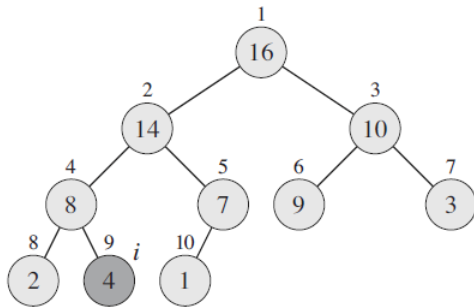
The algorithm ensures the local heap property. The local property (the while loop) translates to a global property.



(a)



(b)



(c)

Example: **Max-Heapify** ($A, 2$)

1. Exchange $A[2]$ with $A[4]$
2. **Max-Heapify** ($A, 4$)
3. Exchange $A[4]$ with $A[9]$
4. **Max-Heapify** ($A, 9$)
5. Done

RUNTIME ANALYSIS

- Call to **MAX-HEAPIFY** costs $O(\log(n))$ due to tree-structure
- **BUILD-MAX-HEAP** calls **MAX-HEAPIFY** $O(n)$ times

Thus, the running time is $O(n \log(n))$. This upper bound is not asymptotically tight.

Time to **MAX-HEAPIFY** depends on the height of the node in the tree. Heights of most nodes are small. An n -element heap

- has height floor $\log(n)$ and
- has at most $\text{ceil} \left[n/2^{h+1} \right]$ nodes of any height h

The time required by **MAX-HEAPIFY** when called on a node of height h is $O(h)$. Now we put everything together.

TOTAL COST OF BUILD-MAX-HEAP

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O \left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h} \right)$$

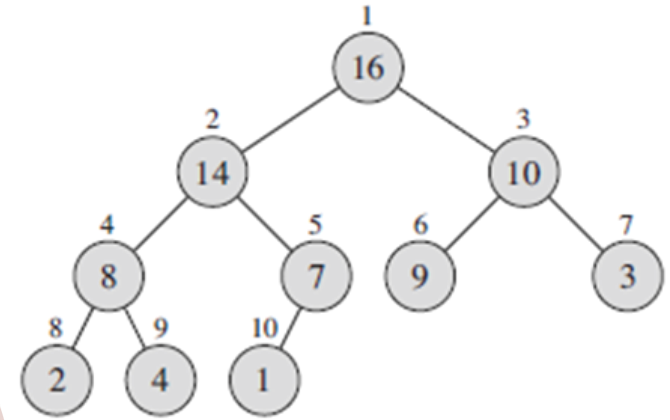
$$\begin{aligned} \sum_{h=0}^{\infty} \frac{h}{2^h} &= \frac{1/2}{(1 - 1/2)^2} \\ &= 2. \end{aligned}$$

$$\begin{aligned} O \left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h} \right) &= O \left(n \sum_{h=0}^{\infty} \frac{h}{2^h} \right) \\ &= O(n). \end{aligned}$$



QUESTIONS ABOUT HEAPS

- What is the height of the given tree?
- What would its representation in array-form look like?
- If a heap corresponds to a tree of height h , how many elements may it maximally have? How many at a minimum?
- If a heap is implemented as an array, at which index is the minimum located?
- If a binary heap contains 126 elements (nodes), what is the height of the corresponding binary tree?



HEAPSORT

HEAPSORT (A)

1. BUILD-MAX-HEAP (A)
2. for $i = A.length$ downto 2
3. exchange $A[1]$ with $A[i]$
4. $A.heap-size = A.heap-size - 1$
5. MAX-HEAPIFY (A, 1)

HeapSort has runtime $O(n \log(n))$.

BINARY EXPONENTIATION

Exponentiating by squaring is a general method for fast computation of large positive integer powers of a number.

ALGORITHM *LeftRightBinaryExponentiation($a, b(n)$)*

//Computes a^n by the left-to-right binary exponentiation algorithm

//Input: A number a and a list $b(n)$ of binary digits $b_I, \dots, b_0$

// in the binary expansion of a positive integer n

//Output: The value of a^n

product $\leftarrow a$

for $i \leftarrow I - 1$ **downto** 0 **do**

product $\leftarrow$ *product* * *product*

if $b_i = 1$ *product* $\leftarrow$ *product* * a

return *product*

Example: Calculate a^{13} using the binary exponentiation algorithm.
Note that $13 = 1101_2$.



In each iteration of the *for*-loop, we have one multiplication, and one test with possible multiplication.

So an upper bound for the number of operations is $4I$, where I is the number of bits in n . That is $O(\log(n))$.

ALGORITHM *LeftRightBinaryExponentiation*($a, b(n)$)

//Computes a^n by the left-to-right binary exponentiation algorithm

//Input: A number a and a list $b(n)$ of binary digits $b_I, \dots, b_0$

// in the binary expansion of a positive integer n

//Output: The value of a^n

$product \leftarrow a$

for $i \leftarrow I - 1$ **downto** 0 **do**

$product \leftarrow product * product$

if $b_i = 1$ $product \leftarrow product * a$

return $product$



SPACE AND TIME TRADE-OFFS

A space – time trade-off is an attempt to trade increased data storage with decreased computation time. Examples include look-up tables instead of calculation of values in real-time, database indexing, rainbow tables, and loop unrolling in program code.



HASHING

A hash function has the following properties:

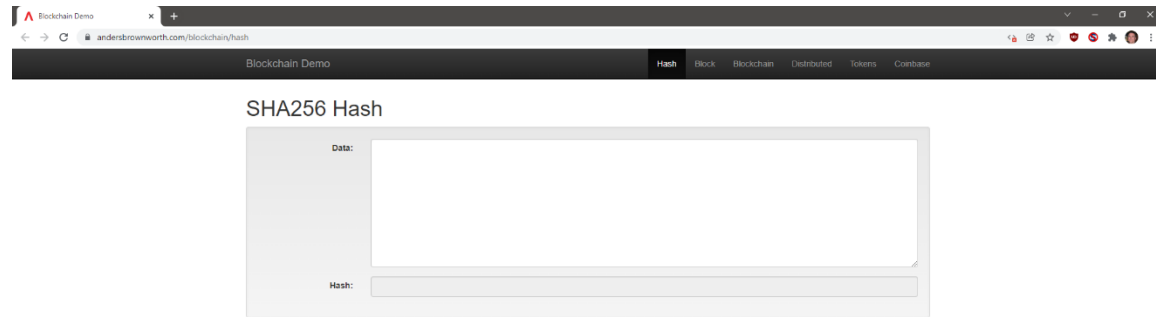
- Its input can be any string (byte array) of any size.
- It produces a fixed size output (e.g., 256-bit output size).
- It is efficiently computable.

Here is a trivial example:

- Each char in the string can be converted to an ASCII code.
- Sum ASCII code of all chars.
- Apply the **mod ()** function on the sum result to get a fixed size output.

CRYPTOGRAPHIC HASH FUNCTIONS

A cryptographic hash function maps data of arbitrary size (*message*) to a bit array of a fixed size (*hash value*, *hash*, or *message digest*).



The screenshot shows a web browser window with the address bar displaying 'andersbrownworth.com/blockchain/hash'. The page title is 'Blockchain Demo'. The navigation bar includes links for 'Hash', 'Block', 'Blockchain', 'Distributed', 'Tokens', and 'Contribute'. The main content area is titled 'SHA256 Hash' and contains a form with two input fields: 'Data:' (a large text area) and 'Hash:' (a single-line text input).

<https://andersbrownworth.com/blockchain/hash>

For a hash function to be cryptographically secure, we are going to require that it has to be collision-resistant.

A collision occurs when two distinct inputs produce the same output. A hash function **Hash ()** is ***collision-resistant*** if nobody can find a collision.

Formally: A hash function Hash is collision resistant if it is *infeasible* to find two values, x and y , such that $x \neq y$, yet $\text{Hash}(x) = \text{Hash}(y)$.

Consider the following hash function:

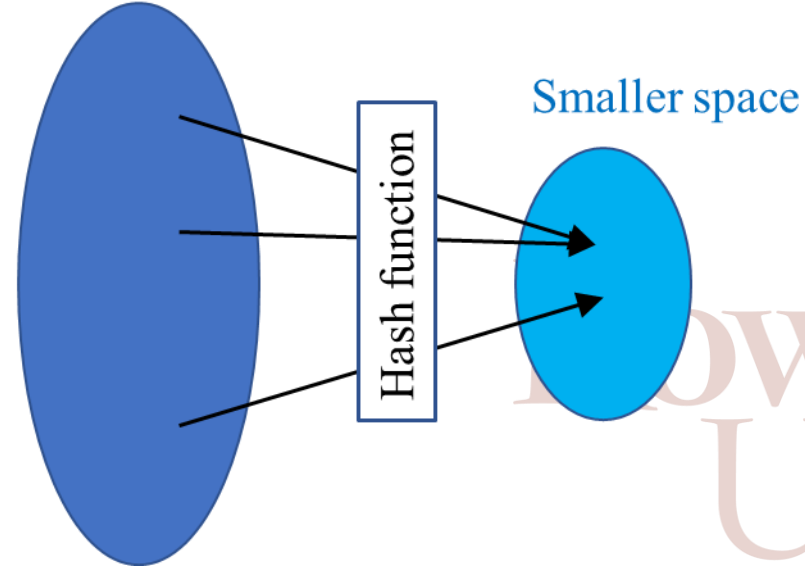
$$\text{Hash}(x) = x \bmod 2^{256}$$

This function meets our requirements of a hash function as it accepts inputs of any length, returns a fixed sized output (256 bits), and is efficiently computable.

However, this function also has an efficient method for finding a collision: One collision then would be the values 3 and $3 + 2^{256}$.

COLLISION RESISTANCE DOES NOT MEAN NO COLLISIONS

Larger space

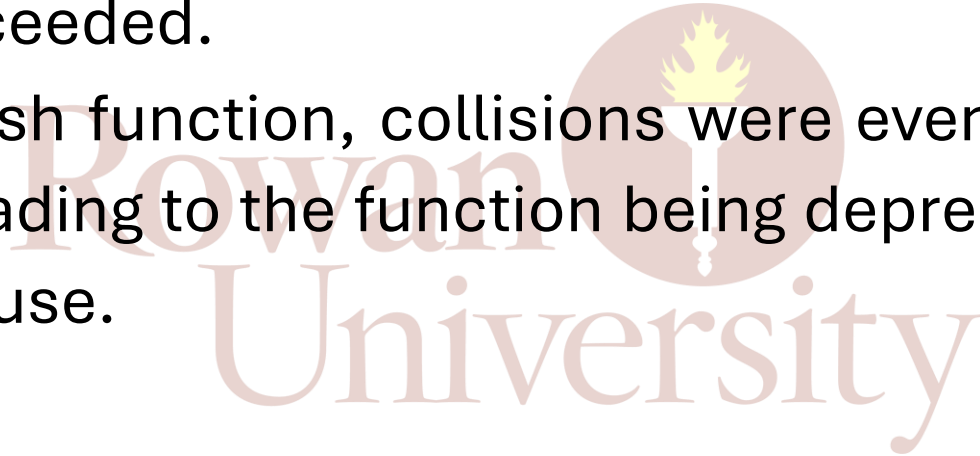


A simple counting argument: The input space to the hash function contains all strings of all lengths, yet the output space contains only strings of a specific fixed length. The input space is larger than the output space, hence there must be input strings that map to

the same output string. By the Pigeonhole Principle there will necessarily be a very large number of possible inputs that map to any particular output.

There are no hash functions proven to be collision-resistant.

- The cryptographic hash functions that we rely on in practice are just functions for which people have tried hard to find collisions and haven't yet succeeded.
- For the MD5 hash function, collisions were eventually found after years of use, leading to the function being deprecated and phased out of practical use.



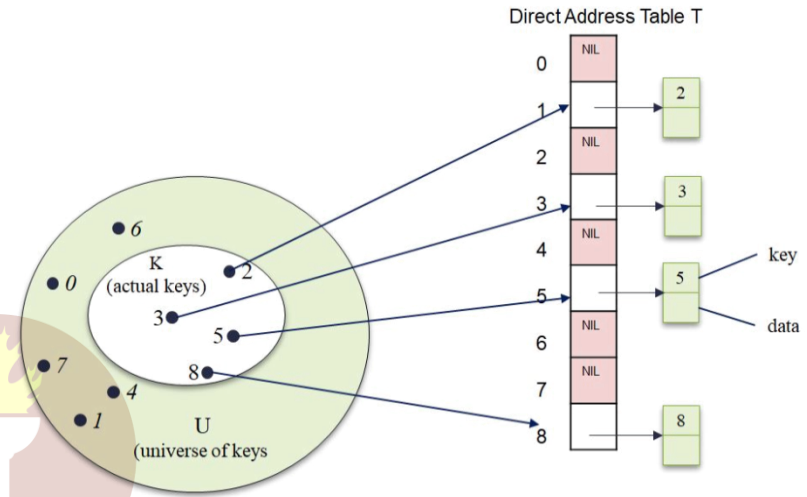
ADT DICTIONARY

Many applications only need the dictionary operations **insert**, **find**, and **delete**. A dynamic set supporting these operations is called ADT Dictionary.

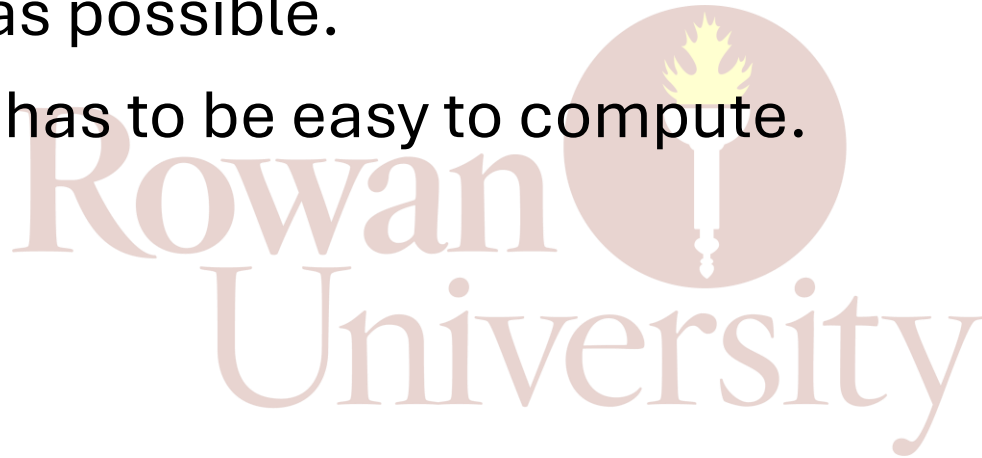
DICTIONARY ()	Create new dictionary.
SEACRH (k)	Find and return value (element) which with key k .
INSERT (k, v)	Insert element v with key k .
DELETE (k)	Delete element that has key k .
ISEMPTY ()	Test whether the dictionary is empty.
CLEAR ()	Delete all elements from the dictionary.

HASH TABLES

We want to combine the simple interface of a dictionary with the fast access of an array. The set of all possible keys in an application may be quite large, though, while the actual keys in use might be a limited quantity. That is the shortcoming of arrays – we will be wasting much index/key space if we reserve an array for all possible keys, but only a relative few are being utilized. A possible solution is to uniformly distribute the keys among a smaller size (the length of the array). Cryptographic hash functions do just that!

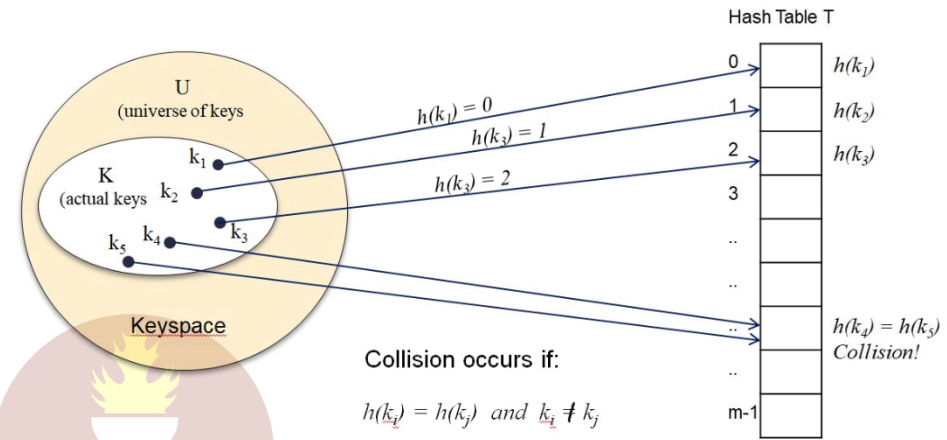


- A hash table's size should not be excessively large compared to the number of keys, but it should be sufficiently large.
- A hash function needs to distribute keys among the cells of the hash table as evenly as possible.
- A hash function has to be easy to compute.



The obvious downside of direct-address tables (arrays) leads us to a two-step process:

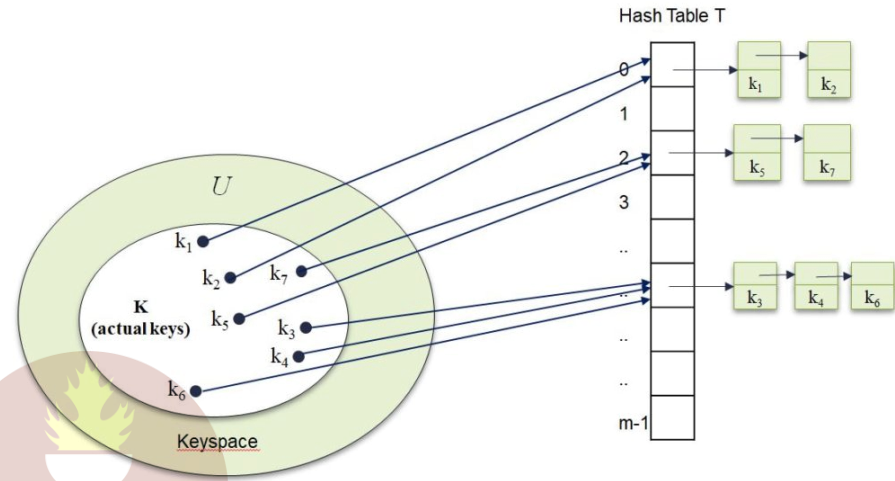
- From key k , calculate a hash value $h(k)$ using a **hash function** h . This function returns numbers within a prescribed range which matches the size of our array.
- Use the hash value $h(k)$ as an index **instead of k** .



This comes at a price: two keys may hash to the same value $h(k_4) = h(k_5)$. This is called **collision**. There are two main methods to deal with this problem: **open hashing** (also called separate **chaining**) and **closed hashing** (also called **open addressing**).

CHAINING

Collisions in hash tables are usually unavoidable, so we need a method to deal with them. In [chaining](#), we place all elements that hash to the same index into a linked list. Instead of a reference to our data in index $i = h(k)$, we store a reference to a linked list containing all elements with hash i . This turns a hash table into a hybrid data structure: array of linked lists. If the hash function distributes n keys among m cells of the hash table about evenly, each list will be about n/m keys long. The ratio $\alpha = n/m$ is called the load factor. It is important for the efficiency of hashing.



OPEN ADDRESSING

In closed hashing, all keys are stored in the hash table itself without the use of linked lists. This implies that the table size m must be at least as large as the number of keys n .

A hash collision is resolved by searching through alternative locations in the array until either the target record is found, or an unused array slot is found, which indicates that there is no such key in the table. The simplest one is called **linear probing**. It checks the cell following the one where the collision occurs. If that cell is empty, the new key is installed there; if the next cell is already occupied, the availability of that cell's immediate successor is checked, and so on. If the end of the hash table is reached, the search is wrapped to the beginning of the table.

Example:

keys	A	FOOL	AND	HIS	MONEY	ARE	SOON	PARTED
hash addresses	1	9	6	10	7	11	11	12

	0	1	2	3	4	5	6	7	8	9	10	11	12
		A											
		A								FOOL			
		A					AND			FOOL			
		A					AND			FOOL	HIS		
		A					AND	MONEY		FOOL	HIS		
		A					AND	MONEY		FOOL	HIS	ARE	
		A					AND	MONEY		FOOL	HIS	ARE	SOON
PARTED		A					AND	MONEY		FOOL	HIS	ARE	SOON

Search and insertion operations are straightforward for this version of hashing, however, deletion is not. We can see that deletion of ARE may pose a problem since SOON (and PARTED as a result) are not where they would be expected.

Let us compare the main properties of hashing with balanced search trees, its main competitor for implementing dictionaries.

- Asymptotic time efficiency
 - With hashing, searching, insertion, and deletion can be implemented to take $\Theta(1)$ time on average but $\Theta(n)$ time in worst case. For balanced search trees, the average time efficiencies are $\Theta(\log(n))$ for both the average and worst cases.
- Ordering preservation
 - Unlike balanced search trees, hashing does not assume key ordering and usually does not preserve it. This makes hashing less suitable for applications that need to iterate over the keys in order or require range queries such as counting the number of keys between some lower and upper bounds.

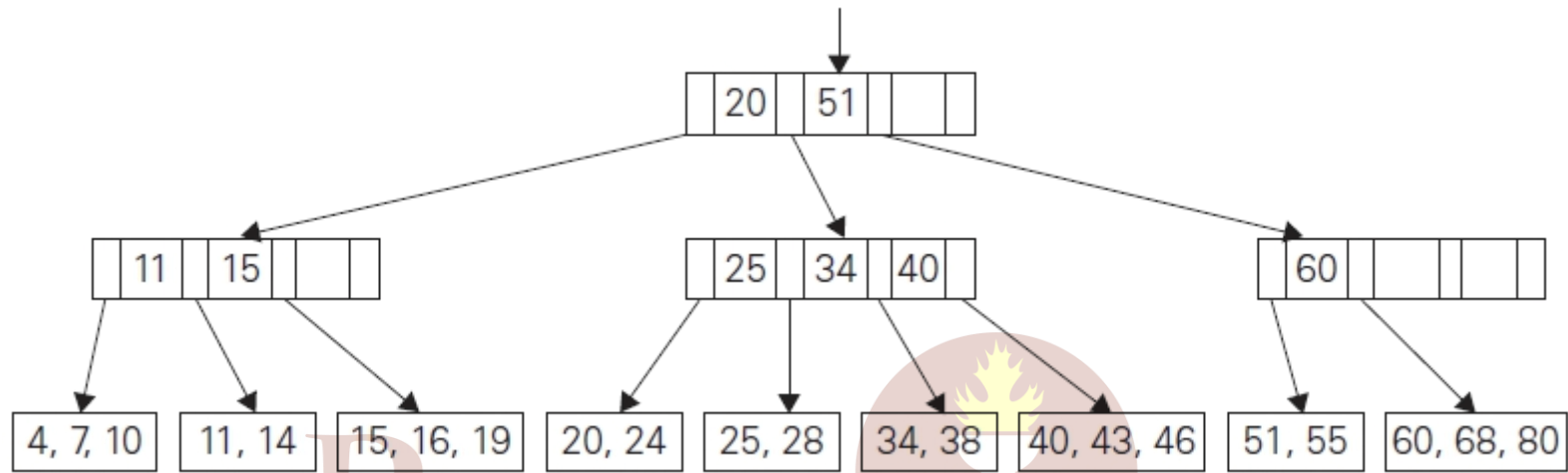
B-TREES

B-trees are another example of self-balancing trees that maintain sorted data and allows searches, sequential access, insertions, and deletions in logarithmic time. The B-tree generalizes the binary search tree, allowing for nodes with more than two children. Unlike other self-balancing binary search trees, the B-tree is well suited for storage systems that read and write relatively large blocks of data, such as databases and file systems.

A B-tree of order m is a tree which satisfies the following properties:

- Every node has at most m children.
- Every node, except for the root and the leaves, has at least $\lceil m/2 \rceil$ children.
- The root node has at least two children unless it is a leaf.
- All leaves appear on the same level.
- A non-leaf node with k children contains $k - 1$ keys.

We see that a 2-3 tree is a B-tree of order 3. Both types were invented in 1970.



Method	Average/Amortized	Worst case
Search	$O(\log(n))$	$O(\log(n))$
Insert	$O(\log(n))$	$O(\log(n))$
Delete	$O(\log(n))$	$O(\log(n))$

QUIZ TOPICS

- Heap terminology
- Array-to-heap and heap-to-array ideas
- Hashing terminology

